IU International University of Applied Sciences

Institutionalizing Clean Code: Strategies for Organizational Implementation

Type of Thesis:	Research Essay
Course Name:	ISSE01_E Seminar: Software Engineering
Course of Study:	Bachelor Software Development
Date:	October 2026
Author:	Tayanana Zifamba
Matriculation Number:	[Insert Number]
Tutor's Name:	Prof. Dr. Carsten Skerra

Table of Contents

List of Abbreviations ..... **III**

1. Introduction **1**

1.1 Relevance and Problem Statement 1

1.2 Research Question, Scope and Approach 1

1.3 Structure of the Essay 2

2. Institutionalizing Clean Code: Strategies and Implementation **2**

2.1 The Theoretical Foundation of Code Quality and Technical Debt 2

2.1.1 Technical Debt from Metaphor to Measurement 3

2.1.2 Clean Code as Principles and as a Shared Standard 3

2.1.3 Refactoring Versus Writing Clean New Code 4

2.2 Technical Processes and Tooling 4

2.2.1 Static Analysis and Quality Gates in CI/CD Pipelines 5

2.2.2 Automated Testing as a Safety Net for Refactoring 5

2.2.3 Code Review as a Social Mechanism 6

2.3 Organizational Culture and Human Factors 6

2.3.1 Management Buy-in and Making Debt Visible 6

2.3.2 Psychological Safety and Shared Ownership 6

2.3.3 Training and Dedicated Time for Refactoring 7

2.4 Critical Assessment of Implementation Barriers 7

2.4.1 Business Realities and Startup Constraints 7

2.4.2 Limits of the Evidence Base 7

2.4.3 Risks of Measurement and Enforcement 8

3. Conclusion **8**

References **10**

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CI	Continuous Integration
CI/CD	Continuous Integration and Continuous Delivery

1. Introduction

1.1 Relevance and Problem Statement

Software development frequently involves a constant trade-off between speed and sustainability. When facing tight project deadlines, developers often rely on quick fixes. While these shortcuts ensure immediate delivery, they make future changes harder, because most software is changed far more often than it is written. Every feature request, bug fix or framework upgrade means that a developer has to read existing code, understand it and change it without breaking something else. Cunningham (1992) described this situation with the metaphor of technical debt: shipping code that is not quite right is like taking out a loan, and the extra effort spent working around that code later is the interest. Since then, the metaphor has grown into its own research field that covers not only source code but also architecture, tests, documentation and build infrastructure (Li et al., 2015). The interest is not just a theoretical idea either. In a longitudinal study of developers' daily work, Besker et al. (2018) found that developers lose a considerable share of their working time because of existing technical debt.

To prevent applications from becoming unmaintainable, teams must prioritize clean coding practices. Martin (2008) describes clean code as code that is easy to read, simple and clear about its intent, and connects it to concrete practices such as meaningful names, small functions and continuous small improvements. Refactoring, in the sense of Fowler (2018), is the technique that moves existing code closer to this ideal without changing its behaviour. However, understanding what clean code looks like is only half the challenge. The larger issue is how a company consistently enforces these standards across different developers, projects and departments. Rachow et al. (2018) report that developers generally accept clean code as valuable, but that this acceptance is not matched by consistent support in their projects, neither from tools nor from the working environment. If clean code only depended on the skills of individual developers, a good training course would solve the problem. The fact that it does not suggests that the organization around the developer plays a much larger role than the clean code literature usually admits.

1.2 Research Question, Scope and Approach

This essay explores the transition from theoretical clean code principles to practical organizational implementation. Its primary objective is to answer the research question: *How can an organization ensure that Clean Coding is applied in practice?* Here, "organization" means any company or team that develops software, from a small startup to a large enterprise. "Ensure" is not understood as a guarantee, because no measure can guarantee that every line of code is clean. Instead, it means creating conditions under which clean code becomes the normal result of everyday work, rather than the personal achievement of a few motivated developers.

The scope is limited to the procedural and cultural mechanisms that enforce code quality, so the essay is not a technical manual on specific programming syntax. It concentrates on code level quality, that is, code debt and design debt that are visible in the source code. Other types of technical debt, such as architectural debt or requirements debt (Li et al., 2015), are only discussed where they directly affect coding practices. Tools such as ESLint, SonarQube, ktlint or detekt are mentioned as examples of a mechanism and not as product recommendations. The practical examples come from Node.js and Express backend development and from Kotlin based Android development in Android Studio, because these are common stacks in modern full-stack and mobile development and show how the same principles look in two different ecosystems.

Methodologically, the essay is a literature based analysis. The core literature consists of three works. Digkas (2021) studied how technical debt develops in open-source projects and whether refactoring old code or writing clean new code is more effective in reducing it. Rachow et al. (2018) examined how clean code is accepted and supported in practice. Vasileva and Schmedding (2016) investigated how code quality can be improved through measurement and refactoring. These works are complemented by research on technical debt management, static analysis, code review, software startups and team psychology. The practical examples make the arguments concrete, but they are illustrations and not empirical evidence.

1.3 Structure of the Essay

To answer the research question, Section 2.1 first establishes the theoretical foundation of code quality and the mechanics of technical debt, including the difference between refactoring existing code and writing clean new code. Section 2.2 then evaluates the technical processes and tooling needed for consistent implementation in modern continuous integration and continuous delivery (CI/CD) pipelines, namely static analysis, quality gates, automated testing and code review. Since these processes only work if people support them, Section 2.3 examines the organizational culture and human factors required to secure buy-in and sustain these practices. Section 2.4 critically assesses the barriers to implementation, including startup constraints and the limits of the literature itself. Finally, Section 3 synthesizes the findings, answers the research question and suggests topics for further research.

2. Institutionalizing Clean Code: Strategies and Implementation

2.1 The Theoretical Foundation of Code Quality and Technical Debt

Before an organization can enforce clean coding practices, it must establish a shared understanding of what constitutes quality code and what the consequences of neglecting it are. This section therefore analyses how the literature defines technical debt and clean code, and what these definitions mean for an organization.

2.1.1 Technical Debt from Metaphor to Measurement

In simple terms, technical debt is the future cost of rework that a team accepts when it chooses a quick and limited solution now instead of a better one that would take longer. Cunningham's (1992) original metaphor was more balanced than the way it is often used today. In his view, a little debt can speed up development, as long as it is paid back promptly once the team understands the problem better. The danger lies in debt that is never repaid, because the interest keeps growing. Technical debt is therefore an economic decision and not only a technical failure. Lim et al. (2012) confirm this from practice, as the practitioners they interviewed often described technical debt as a deliberate trade-off made under business pressure. For an organization, this means that debt is not only created by careless developers but often by decisions about deadlines and priorities.

Kruchten et al. (2012) add that technical debt refers mainly to the invisible parts of a system, such as poor structure, duplicated logic or missing tests, in contrast to visible problems like defects or missing features. This invisibility is one of the main reasons why organizations struggle with debt, because problems that managers and customers cannot see are rarely prioritized. Li et al. (2015) show that the field is still fragmented. Their mapping study identified ten types of technical debt and eight management activities, from identification to repayment, but found that code debt received most of the attention while management and its tool support were less developed.

Digkas (2021) makes the concept measurable at the code level. Using SonarQube, he expresses technical debt as the estimated effort needed to fix rule violations and works with technical debt density, which relates the debt to the size of the code. This normalization matters, because a growing project almost always accumulates more debt in absolute terms, so only the density shows whether quality is actually getting worse. Such a measure also gives organizations a way to estimate the cost of poor code quality and to justify time for refactoring. However, it only captures what static rules can detect. Unclear domain models or misleading names largely escape it, so a low score can create a false sense of security. Tom et al. (2013) support this criticism by naming attitudes, ignorance and oversight as causes of debt, none of which a tool can measure.

2.1.2 Clean Code as Principles and as a Shared Standard

While technical debt describes the problem, clean code describes the target. Martin (2008) defines clean code mainly through its effect on the reader: functions should do one thing, names should reveal intent, duplication should be avoided and the code should be tested. Especially relevant for organizations is the Boy Scout Rule, which says that developers should leave the code a little cleaner than they found it, because it turns clean code into a daily habit. In an Express project, this could mean that a developer who fixes a bug in a long route handler also moves the input validation into a separate, well named middleware function instead of adding another nested if statement.

Compared to technical debt, however, clean code is defined much less precisely. Many of Martin's rules are heuristics that require judgement, and what counts as a "small" function or a "meaningful" name differs between teams and languages. Idiomatic Kotlin with scope functions such as *let* and *apply* may look cryptic to a developer coming from Java. Clean code therefore cannot simply be taken from a book but has to be translated into a shared standard that the team agrees on. Rachow et al. (2018) support this point, since the general acceptance of clean code among the developers they studied did not lead to consistent application, partly because concrete support in daily work was missing. Clean code as a personal value and clean code as an organizational standard are two different things, and the second has to be built actively.

2.1.3 Refactoring Versus Writing Clean New Code

Fowler (2018) defines refactoring as a change to the internal structure of software that makes it easier to understand and cheaper to modify without changing its observable behaviour. He argues that it should happen continuously as part of normal development rather than in separate phases. For organizations with limited time, this raises the question of whether effort should go into cleaning old code or into keeping new code clean.

Digkas (2021) shows that reducing technical debt density is an ongoing balance between these two options. His analysis of Java projects in the Apache ecosystem suggests that developers only repay part of the existing debt, with fixes concentrated on a small set of issue types. At the same time, new code with fewer violations than the existing code base can lower the overall density over time. An organization therefore cannot rely only on occasional large refactoring efforts, since preventing new debt is at least as effective and usually cheaper. Vasileva and Schmedding (2016) reach a similar conclusion, as regular measurement combined with targeted refactoring led to measurable improvements in their work. Feedback about quality has to come early and repeatedly.

There is, however, a tension. A strong focus on new code can neglect legacy hot spots that change often and cause the most interest, such as a fragile authentication module that every developer is afraid to touch. A balanced strategy combines "clean as you code" for every change with deliberate refactoring of the most frequently changed parts. The first part can be largely supported by automation, which is the topic of the next section.

2.2 Technical Processes and Tooling

Enforcing clean code requires embedding quality checks directly into the development lifecycle. Organizations cannot rely solely on developers voluntarily writing clean code, because voluntary practices are the first to be dropped when a deadline gets close. Instead, automated tooling has to act as a first line of defense, in line with Vasileva and Schmedding's (2016) argument that quality improves through continuous measurement and systematic refactoring.

2.2.1 Static Analysis and Quality Gates in CI/CD Pipelines

In a Node.js project, a typical setup includes ESLint with a shared configuration and Prettier for formatting, run locally through a pre-commit hook such as Husky and again in a CI pipeline such as GitHub Actions for every pull request. In Android development with Kotlin, ktlint checks formatting, detekt detects code smells such as long methods or complex conditions, and Android Lint finds platform specific problems, all running as Gradle tasks in Android Studio and on the CI server. SonarQube can then act as a quality gate that automatically rejects a pull request if new code breaks the style guide or exceeds complexity thresholds. A baseline of quality is maintained without anyone checking it manually.

A quality gate on new code is the practical translation of Digkas's (2021) findings. Instead of demanding a perfect legacy code base, the pipeline only demands that each change does not make the debt density worse. This is achievable even with a large legacy system, and it is fair, because developers are only responsible for the code they touch. Vasileva and Schmedding (2016) also show that metrics are most useful when tied to clear goals and regular feedback, and a failed pipeline on a pull request is exactly this kind of immediate feedback.

Static analysis does have limits. Johnson et al. (2013) found that developers often avoid static analysis tools because of false positives, hard to understand warnings and poor workflow integration. A pipeline that floods a pull request with hundreds of warnings on an old file teaches developers to suppress them, the opposite of what was intended. Rachow et al. (2018) likewise identified missing tool support as a barrier. Rule configuration should therefore be an ongoing agreement within the team, and rules nobody believes in should be removed rather than suppressed.

2.2.2 Automated Testing as a Safety Net for Refactoring

Static analysis checks the form of code, but not whether a refactoring broke anything. Since refactoring must not change behaviour (Fowler, 2018), developers need a fast way to check this, otherwise they avoid touching messy but working code and debt accumulates. In an Express API, integration tests with Jest and Supertest call the endpoints and check the responses, so a developer can move business logic into a service layer and know within a minute whether something broke. In Android, JUnit tests for ViewModels and repositories, together with Espresso tests for critical user flows, give the same safety, and they make the automated refactorings in Android Studio, such as Extract Function or Move, much safer to use. Coverage reports also show which modules are untested and risky to change. Coverage is, however, a weak measure of test quality, and when it becomes a strict target, developers may write tests that satisfy the number without protecting behaviour, a problem discussed in Section 2.4.

2.2.3 Code Review as a Social Mechanism

Code review is where technical processes and culture meet. Bacchelli and Bird (2013) found at Microsoft that although finding defects is the main stated motivation, reviews more often lead to better readability, knowledge transfer and team awareness. This makes review the natural place for principles that tools cannot check, for example whether a function name in a Kotlin repository class really describes what it does. When the pipeline already handles formatting, reviewers can focus on design instead of missing semicolons. Review, however, depends on the reviewers' time and attitude, and under pressure it easily turns into quick approvals. Technical tools are therefore necessary but insufficient on their own, which leads to the human factors.

2.3 Organizational Culture and Human Factors

The human element of software engineering largely decides whether clean coding practices succeed or fail. Rachow et al. (2018) emphasize that a major barrier to clean code is the lack of acceptance and support in practical work environments. Three factors address this barrier: management buy-in, psychological safety, and investment in training and refactoring time.

2.3.1 Management Buy-in and Making Debt Visible

If management prioritizes the speed of feature delivery over maintainability, developers will sooner or later abandon clean coding principles, and a quality gate will be switched off the first time it delays an important release. Lim et al. (2012) note that debt decisions are often made under business pressure by people who may not see their long-term cost. This connects to the invisibility of technical debt (Kruchten et al., 2012), because a manager sees shipped features, not code structure. Debt therefore has to be translated into time, cost and risk. The finding of Besker et al. (2018) that a considerable share of developer time is wasted because of debt is exactly this kind of argument. In practice, a team can track debt items in the same backlog as features and estimate them in the same way, or present the SonarQube debt density trend in sprint reviews. This turns debt from a developer complaint into a business decision.

2.3.2 Psychological Safety and Shared Ownership

Clean code also requires developers to be open about weaknesses, since admitting that one's own code is confusing or pointing out that a colleague's module needs refactoring involves interpersonal risk. Edmondson (1999) defines psychological safety as a shared belief that a team is safe for such risk taking and found it to be associated with learning behaviour. Rigorous peer reviews are only possible under these conditions. Where comments are perceived as personal attacks, reviews become vague and developers avoid other people's code, while in a safe team "this name confused me, can we rename it?" is a normal comment. Blame has a similar effect, because if a bug after a refactoring leads to blame, developers stop refactoring. Tom et al. (2013) list low morale as an outcome of debt and attitudes as a cause, which suggests a reinforcing cycle.

Blameless post-mortems, team written coding guidelines and rotating reviewers are cheap ways to break it.

2.3.3 Training and Dedicated Time for Refactoring

Since clean code depends on judgement (Martin, 2008), it can only partly be enforced by rules. Continuous training, such as internal refactoring workshops or pair programming sessions where a junior and a senior developer refactor a class together in Android Studio, spreads this judgement across the team. Many organizations also reserve dedicated time in every sprint for refactoring. Fowler (2018) offers an opposing view, arguing that refactoring should not need a separate slot or justification, because this makes it look optional. Both positions have a point. Small refactorings belong in every estimate, while larger restructurings, such as moving an Android app from very large Activity classes to a Model View ViewModel structure, compete with features for weeks and need dedicated time and an explicit management decision.

2.4 Critical Assessment of Implementation Barriers

Despite the clear benefits of clean code, organizations face legitimate barriers, and the literature itself has blind spots.

2.4.1 Business Realities and Startup Constraints

Giardino et al. (2016) describe how software startups prioritize speed to market and accept technical debt on purpose, because their biggest risk is building something nobody wants. For a small team under extreme financial pressure, strict quality gates may cost more than they save, especially if the product changes direction and the code is thrown away, and strict adherence to clean code could even contribute to business failure. Introducing tooling and retraining developers also requires an upfront investment that a small company may not afford. However, Cunningham (1992) made clear that debt is only acceptable if it is repaid promptly, and startups that survive often end up with a code base that slows them down exactly when they need to scale. A pragmatic strategy is therefore controlled debt with a clear plan to pay it down: linting and formatting from day one, which is cheap in a new Express or Kotlin project, tests for core logic such as payments, and tickets that document deliberate shortcuts. This keeps debt intentional and visible rather than accidental (Lim et al., 2012; Tom et al., 2013). The normative clean code literature (Martin, 2008) says little about such business realities, which is one of its main weaknesses.

2.4.2 Limits of the Evidence Base

The evidence base is also uneven. Digkas (2021) studied open-source projects with voluntary contributors and often strong review cultures, which differ from commercial teams under deadline pressure. Vasileva and Schmedding (2016) worked in an educational context with short project lifetimes, and the survey data of Rachow et al. (2018) shows what developers think rather than

what they do. In addition, the technical debt literature is mainly economic and focuses on measuring and prioritizing debt (Li et al., 2015), while the clean code literature is aimed at the individual (Martin, 2008). Organizations mirror this split by handling debt reactively, through refactoring sprints or rewrites, while clean code is about prevention in every change. Few studies examine how organizational decisions, such as deadlines or incentives, affect code quality at commit level, and Digkas's (2021) focus on new code is only a first step.

2.4.3 Risks of Measurement and Enforcement

Metrics make debt visible for management, but once a metric becomes a target, teams optimize the metric instead of the code by suppressing warnings or writing meaningless tests. Static analysis only captures rule detectable debt (Digkas, 2021; Johnson et al., 2013), while clear names and sensible abstractions depend on judgement, so a green SonarQube dashboard is no proof of clean code. Rigid enforcement can also conflict with psychological safety and produce compliant code but frustrated developers. The literature therefore suggests a layered approach in which automation, review, management support and culture each cover what the others cannot.

3. Conclusion

The objective of this essay was to determine how an organization can ensure that clean coding is applied in practice. The analysis demonstrates that enforcing clean code is not a purely technical challenge but a combination of automated processes and cultural shifts, because the gap between accepting clean code and applying it is mainly caused by the working environment (Rachow et al., 2018). On the technical side, organizations should integrate static analysis, quality gates focused on new code and automated tests into their CI/CD pipelines, for example ESLint and Jest in a Node.js project or ktlint, detekt and JUnit in a Kotlin app. Focusing on new code is realistic and effective (Digkas, 2021), but it has to be combined with deliberate refactoring of frequently changed legacy code. On the human side, code review, management that makes debt visible and provides time for refactoring and training, and a climate of psychological safety are needed to keep these processes alive. Metrics support this work, but they cannot replace the judgement of developers and reviewers.

In conclusion, an organization can only ensure that clean coding is applied in practice by treating code quality as a core business metric rather than an optional engineering preference. At the same time, it should stay pragmatic, configure rules together with the team and accept controlled debt when there is a clear plan to repay it, which is especially relevant for startups. No single layer is sufficient, since tools without culture are bypassed and culture without tools depends too much on the discipline of individuals. These findings are limited by the literature based approach and by an evidence base that comes largely from open-source and educational settings. Future research should therefore examine commercial teams empirically, in particular how deadline pressure, incentives and psychological safety influence the debt density of new code. Another open question

is how the rise of AI code generation affects technical debt density, and whether AI coding assistants help or hinder the enforcement of clean coding standards.

References

Bacchelli, A., & Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. In *2013 35th International Conference on Software Engineering (ICSE)* (pp. 712-721). IEEE. https://doi.org/10.1109/ICSE.2013.6606617

Besker, T., Martini, A., & Bosch, J. (2018). Technical debt cripples software developer productivity: A longitudinal study on developers' daily software development work. In *Proceedings of the 2018 International Conference on Technical Debt* (pp. 105-114). ACM. https://doi.org/10.1145/3194164.3194178

Cunningham, W. (1992). The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications* (pp. 29-30). ACM. https://doi.org/10.1145/157709.157715

Digkas, G. (2021). *Reducing technical debt density: Refactoring vs. writing clean new code* [Doctoral dissertation, University of Groningen]. https://doi.org/10.33612/diss.181245089

Edmondson, A. (1999). Psychological safety and learning behavior in work teams. *Administrative Science Quarterly, 44*(2), 350-383. https://doi.org/10.2307/2666999

Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2nd ed.). Addison-Wesley.

Giardino, C., Paternoster, N., Unterkalmsteiner, M., Gorschek, T., & Abrahamsson, P. (2016). Software development in startup companies: The greenfield startup model. *IEEE Transactions on Software Engineering, 42*(6), 585-604. https://doi.org/10.1109/TSE.2015.2509970

Johnson, B., Song, Y., Murphy-Hill, E., & Bowdidge, R. (2013). Why don't software developers use static analysis tools to find bugs? In *2013 35th International Conference on Software Engineering (ICSE)* (pp. 672-681). IEEE. https://doi.org/10.1109/ICSE.2013.6606613

Kruchten, P., Nord, R. L., & Ozkaya, I. (2012). Technical debt: From metaphor to theory and practice. *IEEE Software, 29*(6), 18-21. https://doi.org/10.1109/MS.2012.167

Li, Z., Avgeriou, P., & Liang, P. (2015). A systematic mapping study on technical debt and its management. *Journal of Systems and Software, 101*, 193-220. https://doi.org/10.1016/j.jss.2014.12.027

Lim, E., Taksande, N., & Seaman, C. (2012). A balancing act: What software practitioners have to say about technical debt. *IEEE Software, 29*(6), 22-27. https://doi.org/10.1109/MS.2012.130

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Rachow, P., Schröder, S., & Riebisch, M. (2018). Missing clean code acceptance and support in practice: An empirical study. In *2018 25th Australasian Software Engineering Conference (ASWEC)* (pp. 131-140). IEEE. https://doi.org/10.1109/ASWEC.2018.00026

Tom, E., Aurum, A., & Vidgen, R. (2013). An exploration of technical debt. *Journal of Systems and Software, 86*(6), 1498-1516. https://doi.org/10.1016/j.jss.2012.12.052

Vasileva, A., & Schmedding, D. (2016). How to improve code quality by measurement and refactoring. In *2016 10th International Conference on the Quality of Information and Communications Technology (QUATIC)* (pp. 131-136). IEEE. https://doi.org/10.1109/QUATIC.2016.034